

Advanced Java MCQ 1

Q1.

```
public class MyServlet extends HttpServlet {  
    public void init() throws ServletException {  
        System.out.println("Servlet Initialized");  
    }  
}
```

What is printed and when?

- A. "Servlet Initialized" every time the servlet is accessed
- B. "Servlet Initialized" only the first time the servlet is loaded
- C. Nothing is printed
- D. Servlet throws an error

Answer: B

The `init()` method is called only once when the servlet is first loaded by the container.

Q2.

```
<%  
    int x = 10;  
    out.println("Value: " + x);  
%>
```

What will be displayed in the browser?

- A. Nothing
- B. Value: 10
- C. Compilation error
- D. out is undefined

Answer: B

`out` is a JSP implicit object. Scriptlets are executed and the result is printed to the browser.

Q3.

Which of the following is the correct order of servlet lifecycle methods?

- A. `doGet()` → `init()` → `destroy()`
- B. `init()` → `service()` → `destroy()`
- C. `init()` → `doGet()` → `destroy()`
- D. `service()` → `init()` → `destroy()`

Answer: B

Servlet lifecycle: `init()` (once), then `service()` (many times), and `destroy()` (once at end).

Q4.

```
<%! int counter = 0; %>
<%= counter++ %>
```

What is the output when this JSP is accessed multiple times?

- A. 0 every time
- B. 1, 2, 3... with each request
- C. Compilation error
- D. Random values

Answer: B

`<%! %>` is a declaration — `counter` is shared across requests (like a class variable).

Q5.

```
@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        response.getWriter().println("Hello");
    }
}
```

What URL pattern maps to this servlet?

- A. `/helloServlet`
- B. `/hello`

- C. /HelloServlet
- D. /

Answer: B

The `@WebServlet("/hello")` annotation maps the servlet to the `/hello` URL.

Q6.

```
<%= 5 + 10 %>
```

What will be displayed?

- A. 5 + 10
- B. 15
- C. Nothing
- D. Syntax error

Answer: B

JSP expression evaluates and prints 15.

Q7.

```
<%= request.getMethod() %>
```

What will this display?

- A. HTTP method like GET or POST
- B. URL
- C. JSP file name
- D. Nothing

Answer: A

`request.getMethod()` returns the HTTP method used for the request.

Q8.

```
public void destroy() {  
    System.out.println("Servlet destroyed");  
}
```

When is `destroy()` called?

- A. Before each request
- B. On servlet creation
- C. When servlet is unloaded or server shuts down
- D. Never

Answer: C

`destroy()` is called when servlet is removed from service.

Q9.

```
<%@ page language="java" contentType="text/html" %>
```

What does this directive affect?

- A. Sets Java version
- B. Changes default response type and language
- C. Declares variables
- D. Imports packages

Answer: B

This page directive sets page-level settings like language and content type.

Q10.

```
ServletContext context = getServletContext();  
context.setAttribute("appName", "MyApp");
```

What is the scope of `appName`?

- A. Request
- B. Session
- C. Application
- D. Page

Answer: C

`ServletContext` stores data for the entire application scope.

Q11.

`${param.name}`

What does this return?

- A. Request parameter "name"
- B. Session attribute
- C. Context param
- D. Cookie value

Answer: A

EL `${param.name}` accesses request parameters.

Q12.

```
RequestDispatcher rd = request.getRequestDispatcher("next.jsp");  
rd.forward(request, response);
```

What happens after this?

- A. Client is redirected
- B. next.jsp is included
- C. Control moves to next.jsp on server
- D. Error occurs

Answer: C

`forward()` sends request internally to another resource.

Q13.

```
<%@ include file="header.jsp" %>
```

What type of include is this?

- A. Dynamic include
- B. Runtime include
- C. Compile-time include
- D. Static import

Answer: C

Include directive adds content at translation/compile time.

Q14.

```
String value = getServletConfig().getInitParameter("param1");
```

Where is `param1` defined?

- A. In JSP
- B. As a class field
- C. In `web.xml`
- D. In the URL

Answer: C

`ServletConfig` reads init parameters defined in `web.xml`.

Q15.

```
<%! int count = 0; %>  
<% count++; %>  
<%= count %>
```

What is the output on first three requests?

- A. 1, 2, 3
- B. 0, 1, 2
- C. 1, 1, 1
- D. Compilation error

Answer: A

Declaration creates shared variable. Scriptlet increments, expression prints it.

Q16. in `web.xml`

```
<servlet>  
    <servlet-name>Test</servlet-name>  
    <servlet-class>com.example.TestServlet</servlet-class>  
</servlet>  
<servlet-mapping>  
    <servlet-name>Test</servlet-name>  
    <url-pattern>/test</url-pattern>  
</servlet-mapping>
```

What URL maps to `TestServlet`?

- A. /TestServlet
- B. /Test
- C. /test
- D. /com/example/TestServlet

Answer: C

URL pattern `/test` maps to the servlet via `web.xml`.

Q17.

```
${sessionScope.user}
```

This expression retrieves `user` from:

- A. Request scope
- B. Application scope
- C. Session scope
- D. Cookie

Answer: C

`sessionScope.user` explicitly refers to session attribute.

Q18.

In servlet, `service()` method is:

- A. Never used
- B. Calls `init()` and `destroy()`
- C. Dispatches request to `doGet()` or `doPost()`
- D. Starts servlet thread

Answer: C

`service()` handles all requests and routes to correct HTTP method handler.

Q19.

```
<jsp:include page="footer.jsp" />
```

What is the effect?

- A. Compile-time inclusion
- B. No inclusion
- C. Runtime inclusion
- D. Header is included

Answer: C

`<jsp:include>` is evaluated at runtime and includes output dynamically.

Q20.

```
<%= session.getId() %>
```

What does this print?

- A. JSP file name
- B. Unique session ID
- C. Server port
- D. Null

Answer: B

`session.getId()` returns unique session identifier.

Q21.

```
<%! public void jspInit() {  
    System.out.println("JSP initialized");  
} %>
```

When is the above method called?

- A. On every request
- B. When JSP is compiled
- C. When JSP is loaded into memory
- D. Never

Answer: C

`jspInit()` is called once when the JSP is loaded, similar to `servlet init()`.

Q22.

```
<%= application.getAttribute("siteName") %>
```

Where is `siteName` stored?

- A. In session scope
- B. In application scope
- C. In request scope
- D. In page scope

Answer: B

The application implicit object refers to ServletContext, i.e., application-wide scope.

Q23.

```
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    response.getWriter().write("Handled by POST");
}
```

When is this method invoked?

- A. On GET request
- B. On POST request
- C. On page load
- D. Never

Answer: B

doPost() handles HTTP POST requests.

Q24.

```
<%@ page errorPage="error.jsp" %>
<%
    int x = 1 / 0;
%>
```

What happens?

- A. Page crashes
- B. error.jsp handles the exception
- C. Compilation error
- D. Arithmetic exception is shown to user

Answer: B

The errorPage directive forwards exceptions to the specified JSP page.

Q25.

```
<jsp:useBean id="user" class="com.model.User" scope="session" />
```

What does this tag do?

- A. Creates a servlet
- B. Declares a method
- C. Instantiates a JavaBean
- D. Initializes an EL variable

Answer: C

`<jsp:useBean>` creates or reuses a JavaBean instance.

Q26.

In `web.xml`:

```
<context-param>  
  <param-name>dbURL</param-name>  
  <param-value>jdbc:mysql://localhost/db</param-value>  
</context-param>
```

Accessed in servlet:

```
String db = getServletContext().getInitParameter("dbURL");
```

What is the value of `db`?

- A. `null`
- B. `jdbc:mysql://localhost/db`
- C. `dbURL`
- D. `localhost`

Answer: B

`getInitParameter()` retrieves context parameters from `web.xml`.

Q27.

Which of these is a correct use of the `page` directive?

- A. `<jsp:page import="java.util.*" />`
- B. `<%@ page import="java.util.*" %>`
- C. `<%! page="java.util.*" %>`
- D. `<page directive="java.util.*" />`

Answer: B

`<%@ page ... %>` is the correct syntax for page directives.

Q28.

Two servlets are mapped:

```
<servlet-mapping>
  <servlet-name>First</servlet-name>
  <url-pattern>/test/*</url-pattern>
</servlet-mapping>
<servlet-mapping>
  <servlet-name>Second</servlet-name>
  <url-pattern>/test/abc</url-pattern>
</servlet-mapping>
```

Which servlet handles `/test/abc`?

- A. First
- B. Second
- C. Both
- D. None

Answer: B

More specific mapping (`/test/abc`) takes precedence over wildcard.

Q29.

What is the purpose of `config`?

- A. Access HTTP headers
- B. Get servlet init parameters
- C. Store session data
- D. Handle cookies

Answer: B

`config` refers to `ServletConfig`, used to get init parameters.

Q30.

```
<%@ page buffer="8kb" %>
```

What does this do?

- A. Sets file upload limit
- B. Sets output buffer size
- C. Limits page size
- D. Controls memory usage

Answer: B

`buffer` attribute defines the response buffer size before output is committed.

Q31.

```
${5 + 3}
```

What is displayed?

- A. 8
- B. 5 + 3
- C. Compilation error
- D. Nothing

Answer: A

EL supports basic arithmetic. It evaluates `${5 + 3}` to 8.

Q32.

Which of the following is true?

- A. `ServletContext` is per servlet
- B. `ServletConfig` is application-wide
- C. `ServletContext` is shared across servlets
- D. Both are request scoped

Answer: C

`ServletContext` is shared across the application; `ServletConfig` is per servlet.

Q33.

Which statement is true about `<%! ... %>`?

- A. It inserts Java code into `service()`
- B. It declares class-level methods/variables
- C. It's used to print output
- D. It's invalid syntax

Answer: B

`<%! ... %>` is for class-level declarations in JSPs.

Q34.

What is the correct JSP lifecycle?

- A. Load → service → destroy
- B. Translate → Compile → Init → Service → Destroy
- C. Compile → Destroy → Load
- D. Init → Load → Compile → Destroy

Answer: B

JSP lifecycle: Translation → Compilation → Initialization (`jspInit`) → Service → Destruction (`jspDestroy`)

Q35.

Difference between `RequestDispatcher.include()` and `forward()`?

- A. `include()` redirects to another page
- B. `forward()` combines outputs
- C. `include()` includes content in response; `forward()` passes control
- D. No difference

Answer: C

`include()` adds content of another resource; `forward()` transfers control completely.

